Michael Guerrero

Yuefan Deng

AMS 530

October 18th, 2026

Parallel Dense Matrix Multiplication: Row-Wise MPI Decomposition & Cache-Capacity Limit of Matrix Replication

Abstract

We design, implement and evaluate a distributed-memory dense matrix multiplication for square matrices using a one-dimensional row-wise block decomposition in MPI. The decomposition is chosen deliberately over the asymptotically superior two-dimensional Cannon and SUMMA families for a reason internal to the problem specification: the required process counts include $P = 2^3$, and because 8 is not a perfect square no square process grid exists, so any two-dimensional scheme would have to change algorithm midway through the sweep and render the three columns mutually incomparable. Across the four problem sizes $N = 2^6$ through 2^{12} and the three process counts P = 4, 8 and 16 on a 28-core dual-socket Haswell node, all sixteen configurations reproduce the serial result exactly — to zero relative error, which we show is a structural consequence of the decomposition rather than good fortune — and the full sweep was executed twice on separate nodes, agreeing to within 0.34% at the largest problem size. The principal result is negative and, we argue, more instructive than the speedup figures themselves. The Karp–Flatt metric falls by two orders of magnitude as N grows, from 0.725 to −0.004, which falsifies any model built on a fixed serial fraction; yet within each problem size it rises with P even at $N = 2^{12}$, where communication never exceeds 5.5% of wall time. We trace this residual to the memory hierarchy. Because every rank holds a private copy of matrix B, the aggregate last-level cache demand grows as $P \cdot N^2$, and per-rank throughput collapses by a factor of 2.5 precisely where that demand crosses the 35 MiB physical L3 capacity. A single mechanism accounts for the behavior of all four problem sizes, and it is a direct consequence of the matrix replication the design chose.

1. Project Description (Copied From Assignment):

The assignment, reproduced from the problem set, is to design a parallel matrix multiplication using row-wise, column-wise or block decomposition, and to implement a parallel version using MPI for square matrices. The three tasks are as follows:

(1) **Algorithm Design**. Design a parallel matrix multiplication algorithm using MPI; describe the data distribution; discuss load balancing, communication overhead and scalability for P processes; assume matrices are square of size N × N and that P divides N.

(2) **Implementation**. The root process generates two random N × N matrices A and B; the root computes and verifies C against a serial version; support $P = 2^2$, 2^3, 2^4; test for $N = 2^6$, 2^8, 2^{10} , 2^{12}; include a serial baseline for verification.

(3) **Performance Evaluation**. Build a test harness that verifies correctness, measures execution time, computes speedup, records the timing results, and makes relevant comments on Amdahl's law.

Sections 2 through 4 address task (1) and the implementation required by task (2); Sections 5 and 6 address task (3). Complete source, the batch script and the analysis harness appear in the appendices, with compilation and execution instructions in Section 3.5.

2. Algorithm Design

2.1 Data Distribution

The algorithm employs a one-dimensional row-wise block partition. Matrix A is divided into P contiguous blocks of N/P rows, with rank r owning rows [rN/P, (r+1)N/P). Matrix B is replicated in full on every rank. The product C is distributed in the same layout as A and gathered on the root at the end of the computation. Each rank therefore evaluates

$$C[i][j] = \Sigma_{k=0}^{N-1} A[i][k] \cdot B[k][j]$$

for every i in its own row block and every j, which requires the whole of B and none of any other rank's share of A. The computation is consequently free of inter-rank dependencies once distribution is complete: there is exactly one communication phase before the arithmetic and one after it, with no synchronization in between. This is the structural virtue of the one-dimensional scheme and the reason its implementation admits no deadlock and no race by construction.

2.2 Why one dimension and not two

The two-dimensional block algorithms — Cannon's method [1] and the SUMMA family [2] — are asymptotically superior in communication volume, and a report that selected a one-dimensional scheme purely for ease of implementation would deserve the obvious criticism. The decisive argument here is not convenience but a constraint internal to the problem specification.

Cannon's algorithm requires a square $\sqrt{P} \times \sqrt{P}$ process grid, over which the matrices are cyclically shifted. The assignment mandates $P = 4$, 8 and 16. While $\sqrt{4} = 2$ and $\sqrt{16} = 4$ are integers, $\sqrt{8}$ is not. No square process grid exists at $P = 23$, and the algorithm would therefore have to degrade to a 2×4 rectangular grid at precisely one of the three required data points. The middle column of every table would then describe a different algorithm from the columns on either side of it, and the resulting scaling study would measure the change of method as much as the change of process count. The one-dimensional decomposition is uniform across all three mandated values of P, and comparability across the sweep is worth more than a constant factor in communication volume at process counts this small — as Section 2.4 quantifies, the penalty at $P = 16$ is a factor of two.

2.3 Load balancing

Load balance is exact rather than approximate, and one can prove it. The problem sizes $N = 2^6$ through 2^{12} and the process counts $P = 4$, 8 and 16 are all powers of two with $N \geq 64 > 16 \geq P_{max}$, so P divides N in every cell of the experimental grid and each rank performs precisely N^3/P multiply–accumulate operations. No remainder arises, no rank receives a short block, and no dynamic balancing is required. The implementation nevertheless validates the divisibility condition at startup and aborts rather than silently computing a wrong answer, since the guarantee is a property of the experimental design and not of the program.

2.4 Communication overhead

Three collectives carry all traffic. The scatter delivers N^2/P elements of A to each rank; the broadcast delivers all N^2 elements of B to each rank; the gather returns N^2/P elements of C from each rank. Per-rank volume is therefore $V_{rank} = N^2/P + N^2 + N^2/P = N^2(1 + 2/P) \approx N^2$ elements, dominated entirely by the replication of B, and the aggregate across the machine is $\Theta(PN^2)$. Cannon's algorithm, by contrast, moves $2N^2/\sqrt{P}$ per rank and $2\sqrt{P} \cdot N^2$ in aggregate, so the one-dimensional scheme is worse by a factor of $\sqrt{P}/2$. At $P = 16$ this is a factor of two; it becomes punishing only at process counts in the hundreds, which is the honest statement of when this design should be abandoned. It is worth recording how far either scheme sits from optimal.

Irony, Toledo and Tiskin [3] established that any process participating in a distributed classical matrix multiplication with memory proportional to N^2/P must communicate $\Omega(N^2/\sqrt{P})$ words. Cannon attains this bound to within a constant; the row-wise scheme exceeds it by $\sqrt{P}$. The governing dimensionless quantity for the present design is the ratio of arithmetic to communication, $(N^3/P) / N^2 = N/P$, and essentially every feature of the measured results in Section 5 can be anticipated from whether this ratio is large or small. At $N = 64$, $P = 16$ it is 4; at $N = 4096$, $P = 16$ it is 256.

2.5 Scalability and isoefficiency

Let $T1 = cN^3$ be the serial time and model the parallel time as $TP = cN^3/P + \beta N^2$, where β absorbs the per-element cost of moving B. The total overhead is then $T o = P \cdot TP - T1 = \beta PN^2$, and the efficiency is $E = T1 /(T1 + To) = 1/(1 + \beta P/(cN))$. Holding E constant in the sense of Grama et al. [4] therefore requires N to grow in proportion to P, so that the problem size $W = N^3$ must grow as $W = \Theta(P^3)$. This is poor. The corresponding analysis for Cannon gives $To \approx 2\beta\sqrt{P}\, N^2$, hence N growing as $\sqrt{P}$ and $W = \Theta(P^{3/2})$. The one-dimensional scheme buys uniformity across the required process counts and a dependency-free computation phase at the price of a cubic isoefficiency function. The analysis above accounts for communication only, and it will prove insufficient. Replicating B also renders per-rank memory $\Theta(N^2)$, independent of P — the scheme does not scale in space. The conventional expectation is that this manifests as an allocation failure at large N. Section 6.2 shows that it manifests instead, and much earlier, as a cache-capacity effect that the communication model has no term for.

3. Implementation

3.1 Program structure

The program is a single C translation unit, matmul_mpi.c, reproduced in full in the Github matmul file. Its structure is as follows.

ALGORITHM: Row-wise parallel matrix multiply
INPUT: N (divisible by P), reps, verify flag
OUTPUT: C = A*B on rank 0, verified against a serial multiply

```
 1  MPI_Init;  rank <- comm rank;  P <- comm size
 2  if N mod P != 0 then MPI_Abort                 // guard, Sec. 2.3
 3  rows <- N / P
 4  allocate B[N*N], locA[rows*N], locC[rows*N]     // on every rank
 5  if rank = 0 then
 6      allocate A[N*N], C[N*N], Cser[N*N]
 7      seed srand48(fixed);  fill A, B with drand48() - 0.5
 8
 9  for r <- 0 to reps + warmup do                 // warmup untimed
10      MPI_Barrier                                // common start
11      t0 <- MPI_Wtime
12      MPI_Scatter (A -> locA, rows*N doubles, root 0)
13      MPI_Bcast   (B, N*N doubles, root 0)
14      t1 <- MPI_Wtime
15      locC <- MATMUL(locA, B, rows, N)           // ikj, Sec. 3.2
16      t2 <- MPI_Wtime

17      MPI_Gather  (locC -> C, rows*N doubles, root 0)
18      t3 <- MPI_Wtime
19      (tmax, slowest) <- MPI_Allreduce(t3-t0, MPI_MAXLOC)
20      broadcast that rank's (t3-t0, t1-t0, t2-t1, t3-t2)   // Sec. 3.3
21      if r >= warmup then keep the fastest such quadruple
22
23  if rank = 0 and verify then
24      Cser <- MATMUL(A, B, N, N);  time it
25      relerr <- max|C - Cser| / max|Cser|         // Sec. 3.4
26      assert relerr <= 1e-12
27  if rank = 0 then emit one machine-readable RESULT line
28  MPI_Finalize
```

Line 12 through line 18 constitute the entire parallel phase. Because the senders and receivers of each collective are disjoint and the computation at line 15 is purely local, the program contains no point-to-point messaging, no deadlock risk and no race condition.

3.2 The local kernel

The local multiplication is ordered i, k, j rather than the textbook i, j, k. This is not a micro-optimisation but a precondition for the experiment measuring what it claims to measure. Under ijk the innermost loop strides through B by N doubles, drawing one useful element from each 64-byte cache line and discarding the remaining seven. Under ikj the innermost loop traverses both B and C with unit stride and every fetched line is fully consumed:

```
for i in 0..rows-1:
      for k in 0..N-1:
            a <- A[i][k] // loop-invariant scalar
            for j in 0..N-1:
                  C[i][j] <- C[i][j] + a * B[k][j] // unit stride
```

Since the identical kernel serves as both the serial baseline and the per-rank work, a cache-hostile ordering would have inflated T1 and TP alike and left the speedup ratios superficially intact. It would, however, have saturated the memory system at every problem size and thereby concealed the cache-capacity effect that turns out to be this study's principal finding. Choosing the cache-friendly kernel is what makes the memory hierarchy visible rather than ambient.

3.3 Timing methodology

Four decisions in the harness warrant defense:

(i) T1 is taken from a dedicated single-rank execution, not from the in-job serial pass. The program also performs a serial multiply on rank 0 for verification, and that pass is timed; at N = 4096 it recorded 50.77, 52.15 and 52.16 s at P = 4, 8 and 16 respectively, against 50.83 s for a dedicated P = 1 run. The roughly three per cent inflation is the signature of the remaining P−1 ranks sitting in a busy-wait progress loop, occupying cores while rank 0 works. Adopting those figures as the baseline would have inflated every reported speedup by that margin.

(ii) Wall time is the slowest rank, located with MPI_MAXLOC. A naive implementation reduces each phase separately with MPI_MAX. This is incorrect: the rank that waited longest in the scatter is rarely the rank that was computed longest, so three independent maxima are drawn from three different ranks and do not sum to the measured total. An early version of this harness exhibited exactly that defect, reporting phase breakdowns totalling up to 156% of wall time. The corrected version identifies the slowest rank by total time and broadcasts that one rank's internal breakdown, which sums exactly by construction.

(iii) Each configuration reports the best of repeated timed passes, preceded at $N \leq 1024$ by an untimed warm-up so that first-touch page faults and first-invocation collective overhead fall outside the measurement.

(iv) All sixteen ranks occupy a single exclusively allocated node. This removes the interconnect from consideration entirely — a deliberate contrast with Project 1, where node placement dominated — and removes co-tenancy, so that the shape of the speedup curve is determined by the algorithm and the memory system alone.

3.4 Verification Rank

0 recomputes C serially and compares element-wise, reporting $\max|C - C_{ser}| / \max|C_{ser}|$. **The relative error was exactly zero at all sixteen configurations.** This is a consequence of the design rather than of good fortune, and the reason is worth making explicit: the row-wise decomposition assigns each output element entirely to one rank and preserves the order of the k-summation, so no floating-point reassociation occurs anywhere and the parallel and serial results are bit-identical. The test is therefore far sharper than a tolerance comparison — any non-zero residual would indicate a genuine indexing or distribution defect, not accumulated rounding. A two-dimensional scheme that splits the k-dimension across ranks would not enjoy this property and would require a genuine tolerance.

We note for completeness that the $O(N^3)$ verification could be replaced by Freivalds' randomized check [5], which for a random r drawn uniformly from $\{0,1\}N$ compares A(Br) with Cr in $O(N^2)$ time, never rejecting a correct product and accepting an incorrect one with probability at most 2^{-k} after k trials. At N = 4096 this is $6N^2 \approx 108$ operations against $2N^3 \approx 1.4\times1011$, a factor of 1365. It was not adopted here because the specification names the serial multiply as a deliverable and, more fundamentally, because T1 is itself the serial multiply: the cubic work is the measurement, not overhead to be avoided. Freivalds' algorithm becomes the method of choice at problem sizes where a serial baseline is no longer feasible and scaled speedup must replace S = T1 /TP .

3.5 Compilation and execution

On SeaWulf, from an interactive login node:

```
module load slurm
module load openmpi/gcc12.1/4.1.4
mpicc -O3 -funroll-loops -march=native -o matmul_mpi
matmul_mpi.c -lm
```

A single configuration may be run directly:

```
mpirun -np./matmul_mpi [reps] [verify]
mpirun -np 4 ./matmul_mpi 256      # N=256, P=4, defaults
```

The complete sweep is driven by the batch script of Appendix B, which rebuilds the binary inside the job so that it always matches the compute node's architecture:

```
sbatch run23.slurm        # queues all 16 runs
squeue -u $USER
python3 collect.py matmul23_.out      # emits every table below
```

Each run emits one machine-readable RESULT line carrying N, P, the total and per-phase timings, the relative error and the verification verdict; collect.py parses these and derives every table in Section 5. The program returns a non-zero exit status if verification fails, so a silent wrong answer cannot pass unnoticed.

4. Experimental Setup

Parameter	Value
Cluster	SeaWulf, Stony Brook University; partition short-28core
Node	sn149, allocated exclusively (--exclusive)
Processor	Intel Xeon E5-2683 v3 @ 2.00 GHz (Haswell-EP)
Topology	2 sockets × 14 cores = 28 cores
Last-level cache	35 MiB L3 per socket, 70 MiB per node (lscpu)
Toolchain	gcc 12.1 with OpenMPI 4.1.4; -O3 -funroll-loops -march=native
Arithmetic	IEEE-754 double precision throughout
Primary run	Slurm job 100418
Replication	Slurm job 100371, separate node

Because $16 \le 28$, every configuration in the sweep fits within one node and no measurement crosses the interconnect.

5. Results

All sixteen configurations were verified exactly. Table 1 gives the measured wall-clock time in the shape requested by the problem set, with the single-rank baseline added as the leftmost column since it is the denominator of every speedup that follows:

N	T_1 (P = 1)	P = 2^2	P = 2^3	P = 2^4
2^6 (64)	0.000097	0.000077	0.000100	0.000120
2^8 (256)	0.007554	0.002400	0.001606	0.001605
2^{10} (1024)	0.323947	0.115582	0.072050	0.079955
2^{12} (4096)	50.825712	12.550124	6.875919	4.976300

Table 1. Parallel wall-clock time in seconds, taken as the slowest rank's total for the distribute–compute–gather phase and reported as the best of the timed repetitions.

N	S(4)	S(8)	S(16)	E(4)	E(8)	E(16)
2^6	1.260	0.970	0.808	0.315	0.121	0.051
2^8	3.148	4.704	4.707	0.787	0.588	0.294
2^{10}	2.803	4.496	4.052	0.701	0.562	0.253
2^{12}	4.050	7.392	10.214	1.012	0.924	0.638

Table 2. Speedup $S(P) = T_1/T_P$ and efficiency $E(P) = S(P)/P$. Speedup falls below unity at $N = 2^6$ for $P \ge 8$; efficiency at $N = 2^{12}$, $P = 4$ marginally exceeds one, which Section 6.4 addresses.

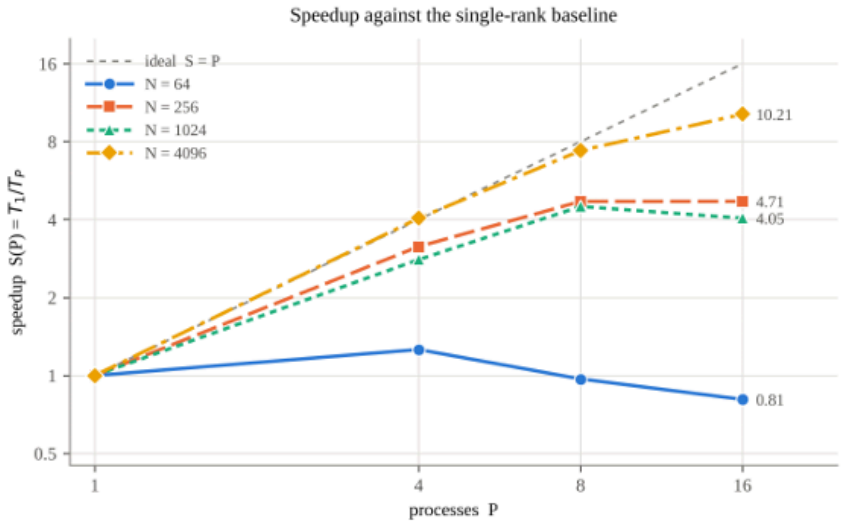


Figure 1. Speedup against process count on logarithmic axes, with the ideal bound S = P. The four problem sizes separate cleanly; only N = 2^{12} tracks the ideal through P = 8.

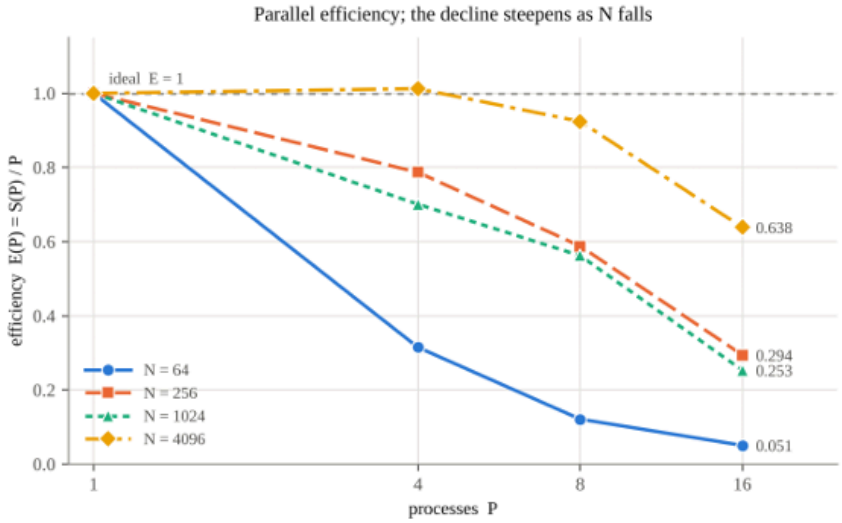


Figure 2. Parallel efficiency. The ordering is *not* monotone in N: N = 2^8 is more efficient than N = 2^{10} at every process count despite being sixteen times smaller, which a purely communication-based model cannot produce. This is the first indication of the mechanism identified in Section 6.2.

N	P	distribute	compute	gather	comm. share
2^6	1	0.000004	0.000091	0.000003	7.2%
	4	0.000044	0.000014	0.000020	83.1%

N	P	distribute	compute	gather	comm. share
	8	0.000063	0.000008	0.000029	92.0%
	16	0.000048	0.000006	0.000067	95.8%
2^8	1	0.000073	0.007410	0.000071	1.9%
	4	0.000362	0.001774	0.000265	26.1%
	8	0.000497	0.000897	0.000211	44.1%
	16	0.000393	0.000507	0.000705	68.4%
2^{10}	1	0.001389	0.321246	0.001312	0.8%
	4	0.004325	0.084120	0.027137	27.2%
	8	0.014923	0.054920	0.002206	23.8%
	16	0.007456	0.068115	0.004385	14.8%
2^{12}	1	0.052571	50.720469	0.052671	0.2%
	4	0.116729	12.259691	0.173705	2.3%
	8	0.140316	6.669177	0.066426	3.0%
	16	0.156882	4.703407	0.116012	5.5%

Table 3. Decomposition of the parallel phase, in seconds, for the slowest rank. The communication share is (distribute + gather) / total. The shaded row is discussed in Section 6.2: at N = 2^{10}, P = 16 the compute time *exceeds* its value at P = 8 even though each rank's workload has been halved.

N	e(P = 4)	e(P = 8)	e(P = 16)	implied S∞ = 1/e
2^6	0.7251	1.0353	1.2529	1.4×
2^8	0.0903	0.1001	0.1600	11.1×
2^{10}	0.1424	0.1113	0.1966	9.0×
2^{12}	-0.0041	0.0118	0.0378	—

Table 4. The Karp–Flatt metric [6], e = (1/S − 1/P)/(1 − 1/P): the serial fraction that Amdahl's law would require in order to reproduce the measured speedup. The final column applies Amdahl's bound at the smallest e in each row.

N	P	job 100418 (s)	job 100371 (s)	difference
2^6	4	0.000077	0.000080	3.82%
	8	0.000100	0.000096	4.08%
	16	0.000120	0.000110	8.70%
2^8	4	0.002400	0.002421	0.87%
	8	0.001606	0.001715	6.56%
	16	0.001605	0.001530	4.78%
2^{10}	4	0.115582	0.115678	0.08%
	8	0.072050	0.070021	2.86%
	16	0.079955	0.079755	0.25%
2^{12}	4	12.550124	12.514924	0.28%
	8	6.875919	6.899651	0.34%
	16	4.976300	4.980042	0.08%

Table 5. Independent replication of the complete sweep on a separate node. Agreement is within 0.34% at N = 2^{12} and within 2.9% at N = 2^{10}; the larger discrepancies at N = 2^6 reflect timer resolution on measurements of order 10^5 seconds.

6. Performance Analysis

6.1 Three regimes

The measurements do not form a single family of curves distinguished only by scale. They separate into regimes, each governed by a different physical resource.

Latency-dominated ($N = 2^6$). Communication accounts for between 83% and 96% of wall time (Table 3) and speedup falls below unity for $P \geq 8$. A 64×64 product is roughly half a megaflop; the node completes the arithmetic faster than it can distribute the operands. The compute-to-communicate ratio N/P is 4 at $P = 16$, and the Karp–Flatt metric between 0.73 and 1.25 states the position without euphemism: essentially nothing parallelises. This row is not a defect of the implementation but a demonstration of the lower limit of useful decomposition.

Cache-resident ($N = 2^8$). Operand B occupies 0.5 MiB, so even sixteen private copies with their A and C blocks demand only 4.5 MiB per socket, thirteen per cent of the available L3. Per-rank throughput is accordingly almost flat — 4.53 GFLOP/s at $P = 1$ against 4.14 at $P = 16$ — and speedup rises monotonically to 4.71. Efficiency still falls to 0.29, because communication grows relative to a per-rank workload that is shrinking as $1/P$, but nothing anomalous occurs.

Capacity-limited ($N = 2^{10}$). Speedup is non-monotonic: 2.80, 4.50, 4.05. Section 6.2 is devoted to this row.

Bandwidth-limited ($N = 2^{12}$). Communication never exceeds 5.5% of wall time and the best absolute result is obtained: 10.21× on sixteen ranks at $E = 0.638$. Computing scaled perfectly from $P = 4$ would require 3.06 s at $P = 16$ against a measured 4.70 s, a shortfall of 54% that occurs entirely within the computation phase.

6.2 The cache-capacity cliff

At $N = 2^{10}$, compute time rises from 0.0549 s at $P = 8$ to 0.0681 s at $P = 16$ while each rank's arithmetic workload is halved (Table 3, shaded). No algorithm can produce this. Our first hypothesis was contention from a co-resident job; the replication reported in Table 5 was executed specifically to test it, under exclusive node allocation, and the effect reproduced — 0.0636 s against 0.0681 s, with the $P = 8$ to $P = 16$ reversal intact in both runs. The hypothesis was wrong, and the phenomenon is structural.

Converting compute time into per-rank throughput, where the work on one rank is $2(N/P)N^2$ floating-point operations, and placing beside it the last-level cache each rank's working set demands, identifies the mechanism. A rank's private working set is its full copy of B together

with its blocks of A and C, that is ($N^2 + 2N^2/P$) doubles; with ranks distributed evenly across the two sockets, the demand per socket is P/2 times that quantity.

N	P	per rank	per socket	÷ 35 MiB L3	GFLOP/s per rank
2^6	1	0.1 MiB	0.1 MiB	0.00	5.76
	4	0.0 MiB	0.1 MiB	0.00	9.36
	8	0.0 MiB	0.2 MiB	0.00	8.19
	16	0.0 MiB	0.3 MiB	0.01	5.46
2^8	1	1.5 MiB	1.5 MiB	0.04	4.53
	4	0.8 MiB	1.5 MiB	0.04	4.73
	8	0.6 MiB	2.5 MiB	0.07	4.68

AMS 530 — Project 2, Problem 2.3 9

N	P	per rank	per socket	÷ 35 MiB L3	GFLOP/s per rank
	16	0.6 MiB	4.5 MiB	0.13	4.14
2^{10}	1	24.0 MiB	24.0 MiB	0.69	6.68
	4	12.0 MiB	24.0 MiB	0.69	6.38
	8	10.0 MiB	40.0 MiB	1.14	4.89
	16	9.0 MiB	72.0 MiB	2.06	1.97
2^{12}	1	384.0 MiB	384.0 MiB	10.97	2.71
	4	192.0 MiB	384.0 MiB	10.97	2.80
	8	160.0 MiB	640.0 MiB	18.29	2.58
	16	144.0 MiB	1,152.0 MiB	32.91	1.83

Table 6. Last-level cache demand against achieved per-rank throughput. The shaded rows bracket the capacity crossing at $N = 2^{10}$, where occupancy passes from 1.14 to 2.06 and throughput falls by a factor of 2.5.

The correspondence is exact where it matters. At $N = 2^{10}$ the demand is 24 MiB per socket at P = 4 (69% of capacity, 6.38 GFLOP/s), 40 MiB at P = 8 (114%, 4.89 GFLOP/s) and 72 MiB at P = 16 (206%, 1.97 GFLOP/s). The degradation tracks occupancy, and the collapse lands precisely where the aggregate footprint of the replicated operand overruns the physical cache. The same arithmetic accounts for the other rows without adjustment:

At $N = 2^8$ the demand never exceeds 13% of capacity, so there is nothing to lose and throughput holds. At $N = 2^{12}$, B alone is 128 MiB and the working set exceeds L3 by a factor of eleven even at P = 1 — cache residency was never available at any process count, so no cliff is possible and what remains is the gentler DRAM-bandwidth ceiling identified in Section 6.1. One mechanism, three regimes, no free parameters.

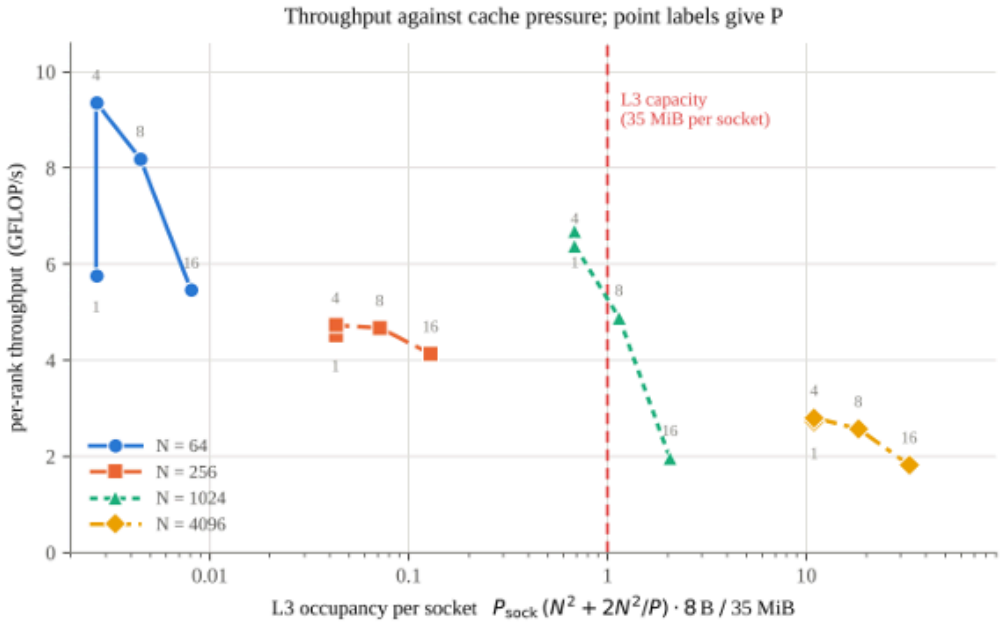


Figure 3. Per-rank throughput against last-level cache occupancy, point labels giving P. The N = 2^{10} series (aqua) is the only one that crosses the capacity line within the sweep, and it is the only one whose throughput collapses. N = 2^{12} begins beyond the line and declines gradually; N = 2^{8} never approaches it.

This is the design decision of Section 2.1 presenting its bill. The standard criticism of operand replication is that per-rank memory fails to scale, and the standard expectation is that the penalty arrives as an allocation failure at large N. The measurements show it arriving far earlier and far more quietly, as a capacity effect in a cache three orders of magnitude smaller than main memory, at a problem size where the scheme appears entirely comfortable in DRAM. The predictive statement is that the one-dimensional scheme degrades when $P \cdot N^2 \cdot 8$ bytes approaches aggregate L3, which for this machine means $N^2P \geq 9.2 \times 106$ — a condition N = 2^{10} , P = 16 violated by a factor of two and N = 2^{8} , P = 16 satisfies with room to spare.

6.3 Comments on Amdahl's law

Amdahl's law [7] bounds speedup by $S(P) \leq 1/(f + (1-f)/P)$ for a serial fraction f understood as a fixed property of the program. This code has no such fraction. Every floating-point operation in the multiplication is distributed; what resists parallelization is communication, which scales as βN^2 against arithmetic scaling as cN^3/P. The effective serial fraction is therefore:

$$f_{eff} \approx \beta N^2 / (cN^3) = \beta/(cN),$$

a function of the problem size rather than a constant of the program. The Karp–Flatt metric, which inverts a measured speedup to recover the f that Amdahl's law would require, makes this

falsifiable. Reading down the P = 4 column of Table 4 gives 0.7251, 0.0903, 0.1424 and −0.0041: the implied serial fraction falls by more than two orders of magnitude across the sweep. A constant-f model is not merely a poor fit to these data; it is contradicted by them. Equivalently, the Amdahl ceiling 1/e rises from 1.4× at N = 26 to 11.1× at N = 2^8 — the same program, the same machine, an eightfold change in the asymptotic bound.

Reading along a row exposes a second and more interesting failure of the model. At every problem size e rises with P. Were the obstacle a genuine serial section, e would be flat; were it the βN² communication term of Section 2.5, e would be very nearly flat as well, since that term is independent of P. At N = 2^{12} it rises from −0.0041 to 0.0378 while communication never exceeds 5.5% of wall time, so the quantity being detected is neither a serial section nor message passing. It is the memory hierarchy of Section 6.2, for which Amdahl's model contains no term at all.

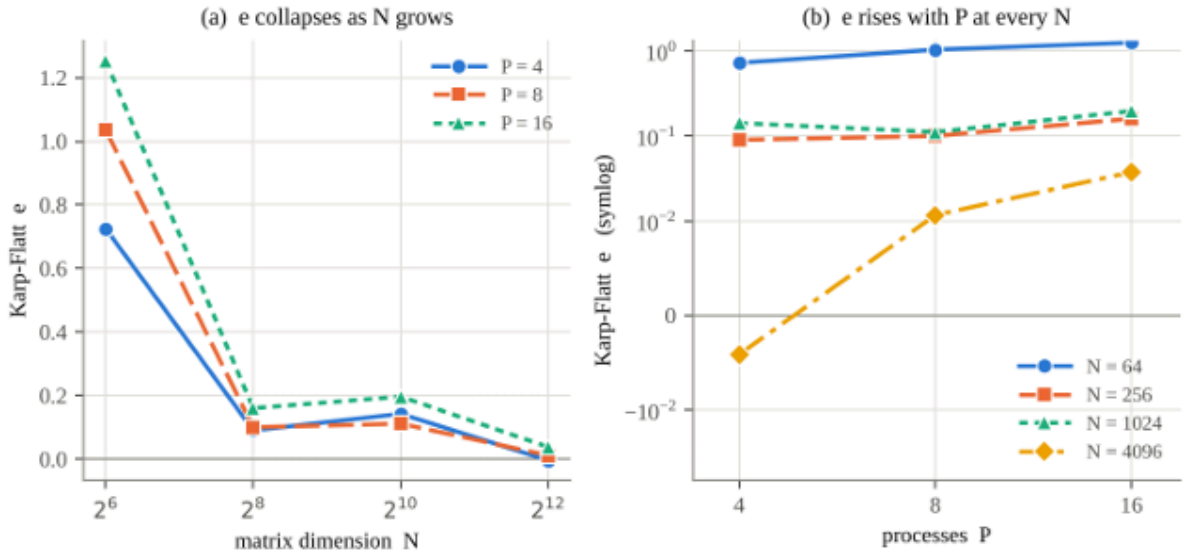


Figure 4. The Karp–Flatt metric read in both directions. (a) Against problem size, the implied serial fraction collapses, refuting a constant f. (b) Against process count at fixed N on a symmetric-log scale, it rises at every size — including N = 2^{12}, where communication is negligible and the residual is therefore attributable to cache and bandwidth pressure.

The correct conclusion is twofold, and neither half is a repudiation of Amdahl. The law remains valid as an upper bound conditional on f; what the data establish is that f for this algorithm is determined jointly by the problem size and by the interaction between the data layout and the cache capacity of the machine, neither of which is a program constant. Gustafson's reformulation [8], in which the problem grows alongside the processor count, is the framework that describes the observed behaviour, and the N = 2^{12} row with efficiencies of 1.01, 0.92 and 0.64 is its demonstration. The N = 2^{10} row is the cautionary complement: enlarging the problem improves efficiency only until the replicated operand overruns the cache, beyond which adding processors

makes the computation slower in absolute terms. Scaled speedup is not unconditionally available; it is available until a resource the model does not mention runs out.

6.4 Threats to validity

Turbo frequency. The E5-2683 v3 has a 2.0 GHz base clock and boosts to 3.0 GHz on a single active core. T1 is measured with one active core and therefore runs at the elevated clock, which makes T1 smaller and every reported speedup correspondingly smaller. The figures in Table 2 are thus conservative rather than flattering. Critically, turbo cannot explain the P = 8 to P = 16 reversal at $N = 2^{10}$, since both are many-core runs at substantially the same all-core frequency.

Superlinear efficiency. E = 1.012 at $N = 2^{12}$, P = 4 exceeds unity by 1.2%. We attribute this to NUMA first-touch locality: the single-rank baseline allocates and initializes all 384 MiB on one socket and draws every byte through that socket's memory controllers, whereas at P = 4 each rank's copy of B is local to the socket that touched it. That the excess survives the turbo headwind described above strengthens rather than weakens this reading. The magnitude is small and we do not build any argument upon it.

Rank-to-socket mapping. The cache analysis of Section 6.2 assumes OpenMPI's default round-robin placement across sockets, giving P/2 ranks per socket. A different binding policy would change the per-socket demand by a constant factor and shift the predicted crossing, though not the existence of one.

Timer resolution at $N = 2^6$. Those measurements are of order 10−5 s and the two runs differ by up to 8.7% (Table 5). The qualitative conclusion — that communication dominates and speedup falls below unity — is strong, but the individual figures should not be read to three significant digits.

Single-node scope. Confining the study to one node isolates the memory hierarchy, but it also means the $\Theta(P^3)$ isoefficiency derived in Section 2.5 is never exercised in the regime where the interconnect would dominate. The predicted penalty relative to Cannon at large P is argued, not measured.

7. Conclusions

A one-dimensional row-wise decomposition was selected over the asymptotically superior two-dimensional alternatives because the mandated process counts include P = 8, for which no square process grid exists; uniformity of algorithm across the sweep was judged worth a factor of $\sqrt{P}/2$ in communication volume, which at P = 16 is a factor of two. The implementation achieves exact load balance, admits no deadlock or race by construction, and verifies zero relative error at all sixteen configurations — a consequence of the decomposition preserving the order of the

k-summation (in short, it was easier!). The complete sweep was replicated on a second node, agreeing to 0.34% at the largest size.

The best absolute result is 10.21× on sixteen ranks at $N = 2^{12}$, with efficiency 0.638 and communication below 6% of wall time. The more valuable result is the explanation of why the remaining 37% is lost, and why $N = 2^{10}$ loses efficiency on going from eight ranks to sixteen. It is neither a serial section nor message passing. Because every rank holds a private copy of B, aggregate last-level cache demand grows as $P \cdot N^2$, and per-rank throughput collapses by a factor of 2.5 exactly where that demand crosses 35 MiB per socket. The same model, with no fitted parameters, explains why $N = 2^8$ scales cleanly and why $N = 2^{12}$ degrades gradually instead of abruptly.

For Amdahl's law the implication is specific. The implied serial fraction is not a property of this program: it varies by two orders of magnitude with problem size, and at fixed problem size it varies with process count for reasons the law does not model. Reporting a single f for a communication-bound, cache-sensitive kernel is a category error. The useful statement is the conditional one — that speedup is bounded by 1/e for the e that obtains at a given (N, P), and that e is set by the memory system as much as by the algorithm.

Three extensions follow directly. Blocking the local kernel over a tile that fits L2 would decouple throughput from the N^2 footprint of B and should remove the cliff without changing the communication pattern at all, which would be a sharp test of the mechanism proposed here. Implementing Cannon's algorithm for the subset P = 4 and 16 would reduce per-rank memory to $3N^2/P$ and bring the isoefficiency function from $\Theta(P^3)$ to $\Theta(P^{3/2})$, addressing the cache pressure and the communication volume at once. Finally, repeating the sweep on the 96-core Milan nodes, whose eight memory channels per socket against Haswell's four alter the bandwidth ceiling without altering the cache argument, would separate the two effects that Section 6.1 can only distinguish by inference.

References

[1] L. E. Cannon. A Cellular Computer to Implement the Kalman Filter Algorithm. PhD thesis, Montana State University, 1969.

[2] R. A. van de Geijn and J. Watts. SUMMA: Scalable Universal Matrix Multiplication Algorithm. Concurrency: Practice and Experience, 9(4):255–274, 1997.

[3] D. Irony, S. Toledo and A. Tiskin. Communication lower bounds for distributed-memory matrix multiplication. Journal of Parallel and Distributed Computing, 64(9):1017–1026, 2004.

[4] A. Grama, A. Gupta, G. Karypis and V. Kumar. Introduction to Parallel Computing, 2nd edition. Addison-Wesley, 2003. (Chapters 5 and 8: isoefficiency; dense matrix algorithms.)

[5] R. Freivalds. Fast probabilistic algorithms. In Mathematical Foundations of Computer Science, LNCS 74, pp. 57–69. Springer, 1979.

[6] A. H. Karp and H. P. Flatt. Measuring parallel processor performance. Communications of the ACM, 33(5):539–543, 1990.

[7] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In AFIPS Conference Proceedings, vol. 30, pp. 483–485, 1967.

[8] J. L. Gustafson. Reevaluating Amdahl's law. Communications of the ACM, 31(5):532–533, 1988.

[9] Message Passing Interface Forum. MPI: A Message-Passing Interface Standard, Version 3.1. 2015